

Real-world (make it feel alive with streaming) — Hints

[Back to the exercise](#) · [Hint 1](#) · [Hint 2](#) · [Solution](#)

Only 2 hints — work through them in order, and don't jump ahead until you've genuinely tried. Each hint has 3 depth levels: **Basic** (the plain idea), **Intermediate** (proper Python), **Advanced** (what a stream chunk actually can and can't guarantee). Read Basic first even if you already know Python — it's the fastest way to spot exactly what each deeper level adds.

- [Hint 1 — The idea, and what each chunk carries](#)
- [Hint 2 — The plan, and almost the whole thing](#)

[Back to the exercise](#) · [Hint 1](#) · [Hint 2](#) · [Solution](#)

Hint 1 — The idea, and what each chunk carries

Basic Version

Add one argument to the same call you already know: `stream=True`. Instead of getting one finished answer back, you get back something you loop over — a series of small pieces of the answer, one at a time, in order.

Use `print(piece, end="")` instead of a normal `print(piece)` — the default `print()` adds a newline after every call, which would put each little piece of text on its own line instead of joining them into one flowing sentence.

[Back to the exercise](#) · [Hint 1](#) · [Hint 2](#) · [Solution](#)

Intermediate Version

With `stream=True`, `client.chat.completions.create(...)` returns an iterable of "chunk" objects instead of one finished response object. Each chunk carries a small piece of the answer at `chunk.choices[0].delta.content` — and this can be `None` (for chunks that don't carry new text, like the very first one), so you need to guard against printing `None`.

`print(text, end="")` — the `end=""` argument replaces the default trailing newline with nothing, so consecutive `print()` calls flow together on the same line, which is what makes streamed output look like it's "typing." Also handle the `None` case cleanly: `piece = chunk.choices[0].delta.content` can be `None`, so check `if piece:` before printing, or use `piece or ""` so you're never trying to print `None` itself.

If you also want to build up the full reply text (useful if you need to save it into conversation history afterward, like in the previous exercise), start with an empty string before the loop and add each piece to it as it arrives.

[Back to the exercise](#) · [Hint 1](#) · [Hint 2](#) · [Solution](#)

Advanced Version

`chunk.choices[0].delta.content` quietly assumes `chunk.choices` always has at least one item in it — but it doesn't always. If you ever turn on usage reporting for a streamed call (`stream_options={"include_usage": True}`), which is how you'd get token counts for a streamed reply, the way `response.usage` gives them to you for a normal call), the *final* chunk in the stream carries the usage numbers and has an **empty** `choices` list — `chunk.choices[0]` on that chunk raises `IndexError`, crashing your loop on the very last chunk, right as the reply was finishing successfully.

There's a second real question worth thinking through, even without writing the code for it here: **what should happen to the partial reply if the connection drops or an error happens mid-stream** — after some words have already been printed to the screen, but before the reply is complete? Do you keep the partial text (so the user sees *something*, even if incomplete), silently discard it, or print a visible "cut off" marker so nobody mistakes a partial answer for a complete one?

The chunk-safety fix is small and worth internalizing:

```
for chunk in stream:
    if not chunk.choices:
        continue # a usage-only chunk with no delta content
    piece = chunk.choices[0].delta.content
    if piece:
        print(piece, end="")
```

Sketch how you'd handle a mid-stream error (a `try/except` around the whole loop, printing whatever text had already accumulated) before checking Hint 2.

Difference between Basic, Intermediate, and Advanced: Basic and Intermediate both stream correctly for the ordinary, uninterrupted case — Intermediate just explains precisely why `end=""` and the `None` guard are needed. Advanced questions the one assumption both earlier levels make without saying so — that every chunk has a `choices[0]` to read `delta` from — and asks what a real caller should do with the words already printed if something goes wrong before the stream finishes, which the tidy happy-path loop never has to consider.

[Back to the exercise](#) · [Hint 1](#) · [Hint 2](#) · [Solution](#)

Hint 2 — The plan, and almost the whole thing

Basic Version

```
call the model with stream=True
for each piece that comes back:
    get the text out of this piece
    if there is text: print it, with no newline after
```

Here's almost the whole thing — just try running it and reading it line by line:

```
stream = client.chat.completions.create(
    model="gpt-4o-mini",
    messages=messages,
    stream=True,
)

for chunk in stream:
    piece = chunk.choices[0].delta.content
    if piece:
        print(piece, end="")

print()
```

Expected output: the reply appears on screen word by word (or a few characters at a time) instead of all at once, then a blank line from the final bare `print()` .

[Back to the exercise](#) · [Hint 1](#) · [Hint 2](#) · [Solution](#)

Intermediate Version

```
function stream_reply(history) -> str:
    stream = call the API with messages=history, stream=True
    full_reply = ""
    for each chunk:
        piece = chunk's delta content
        if piece exists:
            print it, no newline
            add it to full_reply
    print a final newline
    return full_reply
```

```
def stream_reply(history: list) -> str:
    stream = client.chat.completions.create(
        model="gpt-4o-mini",
        messages=history,
        stream=True,
    )

    full_reply = ""
    for chunk in stream:
        piece = chunk.choices[0].delta.content
        if piece:
            print(piece, end="")
            full_reply = full_reply + piece

    print()
    return full_reply
```

`full_reply` accumulates the whole answer as it streams by, so once the loop finishes you have the complete text — useful if you then need to append it to `history` the way the previous exercise did.

[Back to the exercise](#) · [Hint 1](#) · [Hint 2](#) · [Solution](#)

Advanced Version

```
function stream_reply(history) -> str:
    stream = call the API, stream=True
    full_reply = ""
    try:
        for each chunk:
            if chunk has no choices: skip it (usage-only chunk)
            piece = chunk's delta content
            if piece exists: print it, add to full_reply
    except (something went wrong mid-stream):
        print a visible "[cut off]" marker
    print a final newline
    return full_reply    # whatever was accumulated, even if incomplete
```

Turning that into real code — fill in the missing piece yourself:

```
import openai

def stream_reply(history: list) -> str:
    stream = client.chat.completions.create(
        model="gpt-4o-mini",
        messages=history,
        stream=True,
    )

    full_reply = ""
    try:
        for chunk in stream:
            if not chunk.choices:
                continue
            piece = chunk.choices[0].delta.content
            if piece:
                print(piece, end="")
                full_reply += piece
    except openai.APIError:
        # your turn: print a visible marker showing the reply was
        # cut off, so the partial text on screen isn't mistaken
        # for a complete answer
        ...
```

```
print()  
return full_reply
```

Expected behavior: on a normal run, identical output to the Intermediate version. If you simulate a mid-stream failure (hard to trigger on purpose reliably — reasoning through it is the point here), the function still returns whatever text had streamed in so far, instead of losing it or crashing the whole program.

Fill in the `except` block yourself, then compare all 3 of your finished versions against the [Solution](#).

Difference between Basic, Intermediate, and Advanced: Basic and Intermediate both print and (in Intermediate's case) accumulate the reply correctly for a stream that completes normally.

Advanced adds the `chunk.choices` guard (so a usage-only final chunk can't crash the loop with an `IndexError`) and wraps the whole loop so a mid-stream failure leaves you with a clearly-marked partial reply instead of either a silent gap or an unhandled crash — the same "don't let the happy path be the only path you planned for" idea from Hint 1's Advanced question, now written as code.

[Back to the exercise](#) · [Hint 1](#) · [Hint 2](#) · [Solution](#)

Full solution: [Show me the solution](#)